# MA2003B Application of Multivariate Methods in Data Science

## Module 1: Interdependence Relationships


**Instructor:** Raul Gomez
**Email:** rgomez@tec.mx
**Office:** A4-123A


August 19, 2025

# Table of contents

# Part I

# Univariate Normality Testing

# Chapter 1

# An Introduction to the Tidyverse

The Tidyverse is a collection of R packages designed for data science. It provides a consistent and user-friendly interface for data manipulation, visualization, and analysis. Although formally introduced as an ecosystem in 2016, many of these packages were developed by Hadley Wickham between 2007–2014.

## 1.1 Tidy Data

The core idea of behind the design of the Tidyverse is Wickham's definition of "tidy data" [3].

**Definition 1.1.1.** *Tidy Data* is a way of structuring datasets to make them easier to work with. In tidy data:

1. Each variable forms a column.
2. Each observation forms a row.
3. Each type of observational unit forms a table.

This structure allows for easier data manipulation, analysis, and visualization, as it aligns with the way data is typically processed in R. In this section we will describe the basics of the Tidyverse but, for more information on its use, the interested reader can check the following references: [5, 6, 7].

### 1.1.1 Tibbles and pipes

In the Tidyverse, data is often represented using a special type of data frame called a *tibble*. Tibbles are more user-friendly than traditional data frames, as they provide better printing and subsetting behavior. They also allow for more intuitive handling of data. In particular, tibbles are designed to work seamlessly with the pipe operator (`%>%`), which allows for chaining together multiple operations (usually known as "verbs" in the context of the tidyverse) in a clear and concise manner. To illustrate these ideas, in the next section we will perform data cleaning on the "WHO" dataset, to make sure that the associated tibble satisfies the tidy data principles. This is usually the first step in Exploratory Data Analysis (EDA) and is crucial for ensuring that the data is in a suitable format for analysis and visualization.

### 1.1.2 An Example: Tidying the WHO dataset

Before starting, we need to load the required libraries and load the WHO dataset:

```
library("tidyverse")
library("here")
library("cowplot")
library("patchwork")
library("krulRutils")
library("ISLR2")
library("magrittr")

options(scipen = 999) # Disable scientific notation
data(who)
```

We can now start our "tidying" process. The main problem with this dataset is that it contains variables (like "new_sp_m014") that are not very clear and that contain more than one piece of information. So we need to separate these variables and introduce better names for them and their associated values.

```
# We start by creating the variable "who_tidy"
# that contains the cleaned WHO dataset
who_tidy <- who %>%
  # We use pivot_longer to eliminate the variables starting with "new"
  # and use them as values instead
  pivot_longer(
    cols = starts_with("new"),
    names_to = "key",
    values_to = "cases",
    values_drop_na = TRUE
  ) %>%
  mutate(
    key = if_else(
      startsWith(key, "newrel"),
      sub("newrel", "new_rel", key),
      key
    ),
    cases = as.integer(cases)
  ) %>%
  separate(key, into = c("new", "type", "sexage"), sep = "_") %>%
  separate(sexage, into = c("sex", "age"), sep = 1) %>%
  select(-new, -iso2, - iso3)  %T>%
  # Finally, we save the tidy data in both RDS and CSV formats
  saveRDS(here("data", "who_tidy.rds")) %>%
  write_csv(here("data", "who_tidy.csv")) %>%
  glimpse()
```

```
Rows: 76,046
```

```
Columns: 6
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

We now want to convert the columns “type”, “sex”, and “age” into factors. We start by constructing the following lookup tables:

```r
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "Extrapulmonary TB",
    "Relapse case",
    "Smear-Negative pulmonary TB",
    "Smear-Positive pulmonary TB"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Female", "Male")
)

who_age_lookup_tbl <- tibble(
  code = c(
    "014",
    "1524",
    "2534",
    "3544",
    "4554",
    "5564",
    "65"
  ),
  label = c(
    "0-14",
    "15-24",
    "25-34",
    "35-44",
    "45-54",
    "55-64",
    "65+"
  )
)
```

We can now append factor columns for the corresponding variables in the WHO dataset:

```
who_factor <- who_tidy %>%
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
  ) %>%
  glimpse()
```

```
Rows: 76,046
Columns: 9
$ country     <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan"~
$ year        <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997~
$ type        <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp"~
$ sex         <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f"~
$ age         <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014"~
$ cases       <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128~
$ type_factor <fct> Smear-Positive pulmonary TB, Smear-Positive pulmonary TB, ~
$ sex_factor  <fct> Male, Male, Male, Male, Male, Male, Male, Female, Female, ~
$ age_factor  <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24,~
```

Finally, we want to use this information to analyze the number of cases of tuberculosis by type and sex. We start by generating the corresponding frequency tibble.

```
who_type_sex_tbl <- who_factor %>%
  count(type_factor, sex_factor, wt = cases, name = "cases") %>%
  print(n = Inf)
```

```
# A tibble: 8 x 3
  type_factor                 sex_factor    cases
  <fct>                       <fct>         <int>
1 Extrapulmonary TB           Female       941880
2 Extrapulmonary TB           Male        1044299
3 Relapse case                Female      1201596
```

```
4 Relapse case                Male        2018976
5 Smear-Negative pulmonary TB Female      2439139
6 Smear-Negative pulmonary TB Male        3840388
7 Smear-Positive pulmonary TB Female     11324409
8 Smear-Positive pulmonary TB Male       20586831
```

## 1.2 The "Grammar of Graphics" and ggplot2

The concept of the "Grammar of Graphics" was introduced by Leland Wilkinson in his 1999 book *The Grammar of Graphics* [8]. This provides a framework for understanding how to create visualizations in a systematic way. It breaks down the process of creating a plot into components such as data, aesthetics, geometries, statistics, coordinates, and themes. The `ggplot2` package [4] implements this grammar in R, allowing users to build complex visualizations by layering these components. For more information on the Grammar of Graphics, its history and applications, the interested reader can consult the following references: [1, 2].

### 1.2.1 Layers in ggplot2

In `ggplot2`, plots are constructed by adding multiple *layers* that define the components of the visualization. Each layer corresponds to a specific aspect of the plot, and together they form the complete graphic. These layers include:

1. **Data:** The dataset to be visualized (usually a data frame or tibble). This is the source of the information for the plot.
2. **Aesthetics:** Mappings that relate variables in the data to visual properties of the plot, such as:
   - Position on the x- and y-axes.
   - Color, fill.
   - Size, shape.
   - Transparency.

   These mappings define *what* data is shown and *how* it is represented visually.
3. **Geometries:** Geometric objects that display the data, for example:
   - `geom_point()` for scatterplots.
   - `geom_line()` for line graphs.
   - `geom_col()` for bar charts.
   - `geom_histogram()` for histograms.

   The geometry controls *how* the data is drawn.
4. **Statistical transformations:** Optional calculations applied to the data before plotting, such as:

- `stat_bin()` for binning data in histograms.
- `stat_smooth()` for fitting smooth curves.

These are preprocessing steps for the data visualization.

5. **Scales:** Define how data values are translated into visual properties, for example mapping numeric values to colors or shapes. Scales also control axis ticks, legends, and guides.
6. **Coordinates:** The coordinate system used for the plot, such as Cartesian (`coord_cartesian()`), polar (`coord_polar()`), or flipped coordinates (`coord_flip()`).
7. **Facets:** Methods to split the data into subsets and display multiple plots arranged in a grid:
   - `facet_wrap()`
   - `facet_grid()`
8. **Labels:** `labs()` adds titles, axis labels, and captions.
9. **Themes:** `theme()` controls the overall appearance of the plot (fonts, background, gridlines).

Each layer can be combined and customized to build complex and elegant plots. This *layered grammar* allows you to think of your graphic as a composition of independent components rather than a single, monolithic object.

### 1.2.2 Plotting with ggplot2

We will illustrate these concepts by plotting the number of cases of tuberculosis by type and sex, using the frequency tibble we created earlier.

```
who_type_sex_plot <- who_type_sex_tbl %>%
  ggplot(aes(x = type_factor, y = cases, fill = sex_factor)) +
  geom_col(position = "dodge") +
  c_scale_fill("C rose", "C blue") +
  labs(
    title = "Tuberculosis Cases by Type and Sex",
    x = "Tuberculosis Type",
    y = "Cases",
    fill = "Sex"
  ) +
  theme_krul()
```

We can now save and plot the graph:

```
ggsave(
  filename = here("images", "who_plot.png"),
  plot = who_type_sex_plot,
  width = 12,
  height = 6,
  dpi = 300
```

```
)
who_type_sex_plot
```

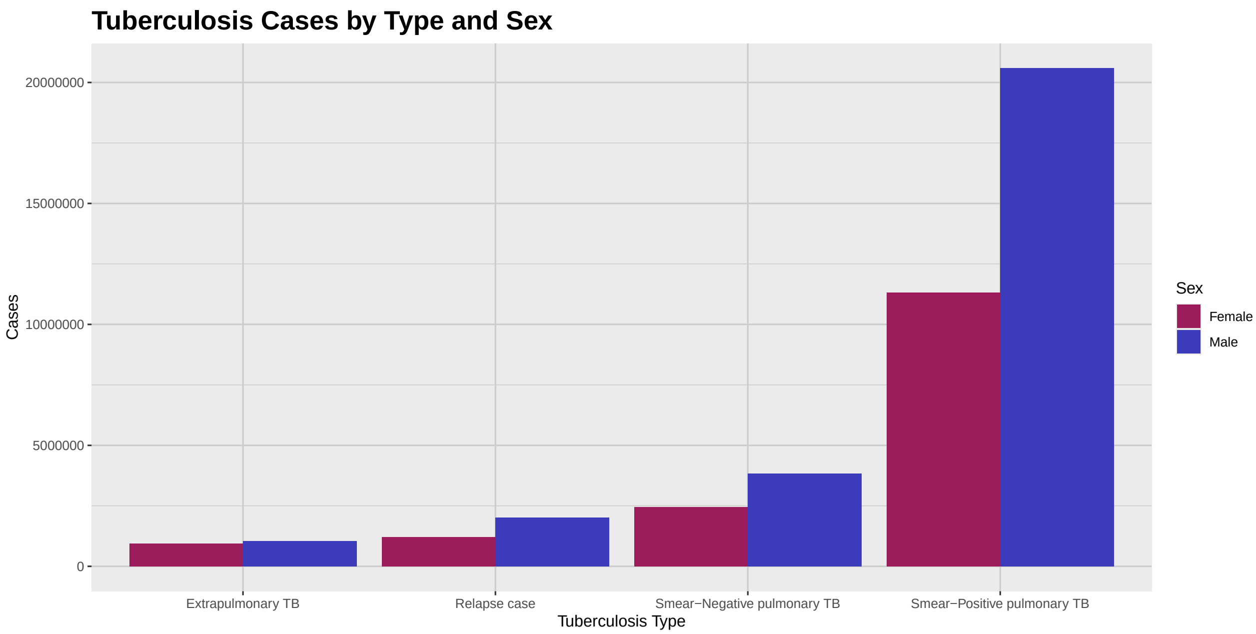


Figure 1.1: This plot shows a comparative analysis of the number of cases of tuberculosis by type and sex. As we can see from the plot, the number of cases is consistently higher for the male population across all types of tuberculosis.

## 1.3 Activity: Dataset Tidying

1. In the previous section we showed how to use the `%>%` operator to chain together multiple operations in the Tidyverse. In this exercise, you should show the result of all the intermediate steps in the tidying process of the WHO dataset, using the pipe operator.
2. The problem with the Billboard dataset, according to the principles of "tidy data", is that the weeks in which a song appeared in the Billboard chart are represented as columns, which makes it difficult to analyze the data. Using the techniques we have learned so far, the reader is required to tidy the Billboard dataset, so that it can be used to analyze the evolution of the ranking of the song "Who Let The Dogs Out" by "Baha Men" in the Billboard Hot 100 chart. The final visualization should look like the one shown below.

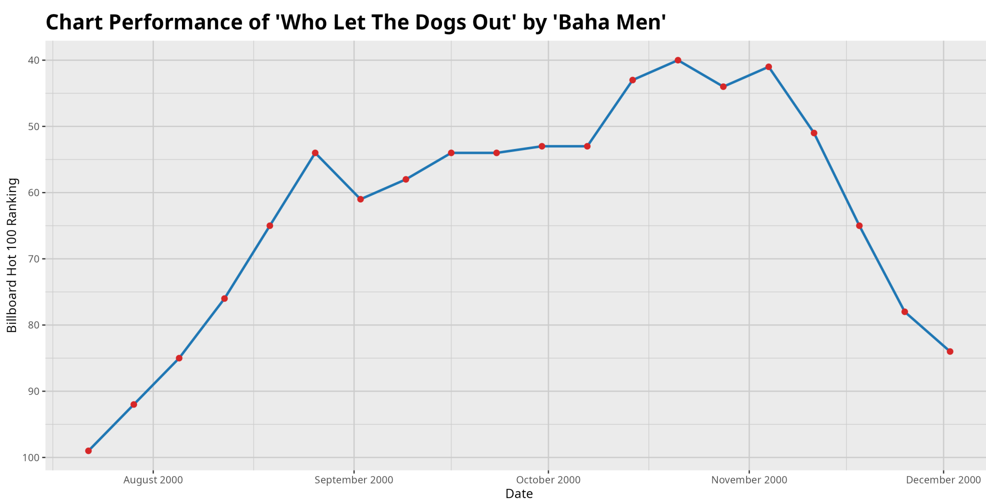


Figure 1.2: Evolution of the ranking of the song ’Who Let The Dogs Out’ by ’Baha Men’ in the Billboard Hot 100 chart. The song peaked at rank 40 in the week of October 21st, 2000, and remained in the top 100 for several weeks.

# Bibliography


[1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.

[2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. https://otexts.com/fpp3/

[3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, *59*(10), 1–23. https://doi.org/10.18637/jss.v059.i10

[4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.

[5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O'Reilly Media.

[6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.

[7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, *4*(43), 1686. https://doi.org/10.21105/joss.01686

[8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

# Chapter 2

# The Standardized Moments

As it is well known, any normal distribution is completely determined by its mean and standard deviation. However, many real-world datasets do not follow a normal distribution. In this chapter, we will explore some other distributions and show how the concepts of skewness and kurtosis can help us understand its shape. We will also introduce QQ plots, which are useful graphical tools for comparing the distribution of a dataset against a theoretical distribution, which is usually taken to be normal. But before starting, let's load the necessary libraries and set some options.

```r
library(tidyverse)
library(broom)
library(here)
library(patchwork)
library(krulRutils)
library(magrittr)
library(e1071) # for skewness and kurtosis functions
library(car)

options(scipen = 999)
```

## 2.1 Skewness

**Definition 2.1.1.** The *skewness* of a random variable $X$ is defined as the third standardized moment, given by

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X-\mu)^3]}{\sigma^3},$$

where $\mu = \mathbb{E}[X]$ is the mean and $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ is the standard deviation of $X$.

Skewness measures the asymmetry of a probability distribution. A distribution is said to be *positively skewed* (or right-skewed) if it has a longer tail on the right side, and *negatively skewed* (or left-skewed) if it has a longer tail on the left side. A perfectly symmetric distribution has a skewness of zero. To illustrate this idea, we will introduce the gamma distributions.

**Definition 2.1.2.** We say that a continuous random variable $X$ follows a *Gamma distribution* with shape parameter $\alpha > 0$ and rate parameter $\beta > 0$, if its associated probability density function (PDF) is given by

$$f_X(x;\alpha,\beta) = \begin{cases} \dfrac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \tag{2.1}$$

where $\Gamma(\alpha)$ denotes the Gamma function, defined as

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t}\, dt.$$

Among other applications, the Gamma distribution is frequently used to model the number of total insurance claims over a given time period.

*Remark* 2.1.3. The skewness of a Gamma distribution is given by

$$\gamma_1 = \frac{2}{\sqrt{\alpha}}.$$

This means that the skewness is always positive, and it approaches zero as $\alpha$ increases. In other words, the distribution becomes more symmetric as $\alpha$ becomes larger.

We will now show how to plot the PDF of the Gamma distribution for $\alpha = 2$ and $\beta = 1$, highlighting the mean and median.

```r
# Parameters
alpha <- 2
beta <- 1

# Create sequence of x values
x_data <- seq(0, 10, length.out = 200)

# Calculate density values
gamma_distribution_tbl <- tibble(
  x_data = x_data,
  density = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculate mean and median
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
gamma_sd <- sqrt(alpha) / beta

# Plot
gamma_distribution_plot <- gamma_distribution_tbl %>%
ggplot(aes(x = x_data, y = density)) +
  geom_line(
```

```
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = gamma_mean,
    y = 0.5,
    label = "Mean",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = gamma_median,
    y = 0.5,
    label = "Median",
    color = c_palette["C green"],
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "Gamma Distribution PDF (shape =", alpha, ", rate =", beta, ")"
    ),
    x = "X",
    y = "Density"
  ) +
  theme_krul()

gamma_distribution_plot
```

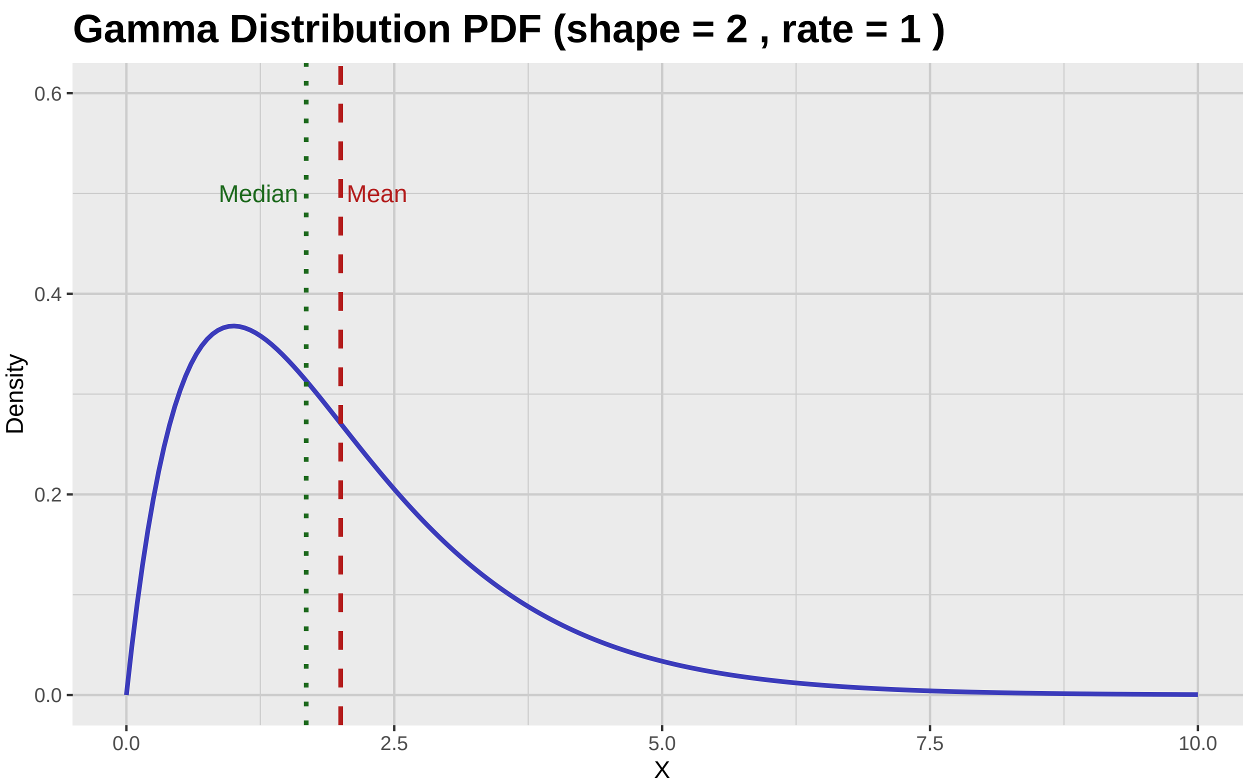


Figure 2.1: PDF of the gamma distribution with mean and median highlighted.

## 2.2 Kurtosis

**Definition 2.2.1.** The *kurtosis* of a random variable $X$ is defined as the fourth standardized moment, given by

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X-\mu)^4]}{\sigma^4},$$

where $\mu = \mathbb{E}[X]$ is the mean and $\sigma = \sqrt{\mathbb{E}[(X-\mu)^2]}$ is the standard deviation of $X$.

Kurtosis measures the "tailedness" of a probability distribution. A distribution with high kurtosis (also known as a *leptokurtic* distribution) has heavier tails and a sharper peak than a normal distribution, while a distribution with low kurtosis (also known as a *platykurtic* distribution) has lighter tails and a flatter peak. The kurtosis of a normal distribution is 3, which is why we are usually interested in computing the *excess kurtosis* which is given by:

$$\gamma_2 = \beta_2 - 3.$$

To illustrate this idea, we will introduce the Laplace distribution.

**Definition 2.2.2.** We say that a continuous random variable $X$ follows a *Laplace distribution* with location parameter $\mu \in \mathbb{R}$ and scale parameter $b > 0$, if its associated probability density function (PDF) is given by

$$f_X(x; \mu, b) = \frac{1}{2b} \exp\left(-\frac{|x-\mu|}{b}\right), \quad x \in \mathbb{R}. \tag{2.2}$$

Among other applications, the Laplace distribution is often used in image processing and computer vision, particularly in modeling noise in images.

*Remark* 2.2.3. The excess kurtosis of a Laplace distribution is given by

$$\gamma_2 = 3.$$

This means that the Laplace distribution has heavier tails than a normal distribution.

We will now show how to plot the PDF of the Laplace distribution with mean zero and variance one.

```r
# Parameters
mu <- 0      # mean (location)
b <- 1/sqrt(2)       # scale

# Sequence of x values covering enough range to see tails
x_data <- seq(-5, 5, length.out = 300)

# Laplace PDF function
dlaplace <- function(x, mu, b) {
  return(1/(2*b) * exp(-abs(x - mu)/b))
}

# Create data frame with PDF values
laplace_distribution_tbl<- tibble(
  x_data = x_data,
  density = dlaplace(x_data, mu, b)
)

# Plot PDF
laplace_distribution_tbl %>%
  ggplot(aes(x = x_data, y = density)) +
  geom_line(color = c_palette["C blue"], linewidth = 1) +
  labs(
    title = "Laplace Distribution PDF (mean = 0, var = 1)",
    x = "x",
    y = "Density"
  ) +
  theme_krul()
```

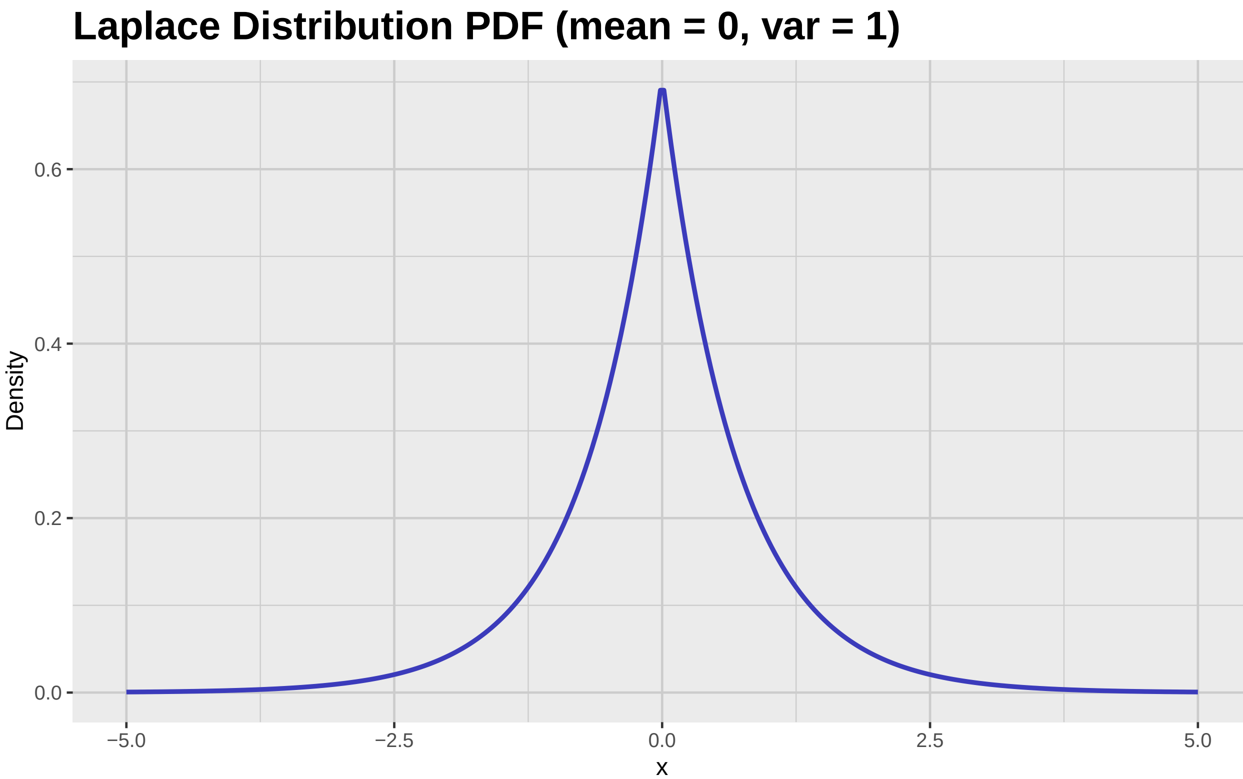


Figure 2.2: PDF of the Laplace distribution with mean zero and variance one.

## 2.3 QQ Plots

QQ plots, or quantile-quantile plots, are graphical tools used to compare the distribution of a dataset against a theoretical distribution, such as the normal distribution. They are particularly useful for assessing the normality of the data.

To construct a QQ plot, you should follow these steps:

1. **Sort the sample data:** Arrange the data in increasing order:

$$x_{(1)} \leq x_{(2)} \leq \cdots \leq x_{(n)}.$$

2. **Calculate plotting positions (empirical quantiles):** For each ordered value $x_{(i)}$, compute the corresponding probability

$$p_i = \frac{2i-1}{2n}, \quad i = 1, 2, \ldots, n.$$

3. **Find theoretical quantiles of the normal distribution:** Compute the theoretical quantiles from the inverse cumulative distribution function (CDF) of the normal distribution:

$$q_i = \Phi^{-1}(p_i),$$

where $\Phi^{-1}$ is the quantile function of the standard normal distribution.

4. **Plot the points:** Plot the pairs $(q_i, x_{(i)})$ with $q_i$ on the x-axis and $x_{(i)}$ on the y-axis.
5. **Add a reference line:** Typically, we add a line through the first and third quartiles to help assess normality.

The closer the points lie to this line, the more the sample resembles the specified normal distribution.

To illustrate these ideas, we will now generate samples from the Gamma and Laplace distributions, and then we will construct their associated QQ plots and histograms. First we construct the samples:

```
set.seed(123)  # for reproducibility

# Sample size
n <- 300
# Generate random samples from the Gamma distribution
gamma_sample <- rgamma(n, shape = alpha, rate = beta)

gamma_sample_tbl <- tibble(
  sample = gamma_sample
)

# Generate uniform random numbers in (0,1)
u <- runif(n)

# Apply inverse CDF of Laplace
laplace_sample <- ifelse(u < 0.5,
                         mu + b * log(2*u),
                         mu - b * log(2*(1 - u)))

laplace_sample_tbl <- tibble(
  sample = laplace_sample
)
```

We will now construct the corresponding QQ plots and histograms. We will start with the Gamma distribution samples:

```
gamma_qq_plot <- gamma_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plot",
```

```
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

gamma_histogram <- gamma_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

gamma_plot <- gamma_qq_plot + gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gamma Distribution: QQ Plot and Histogram",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

gamma_plot
```

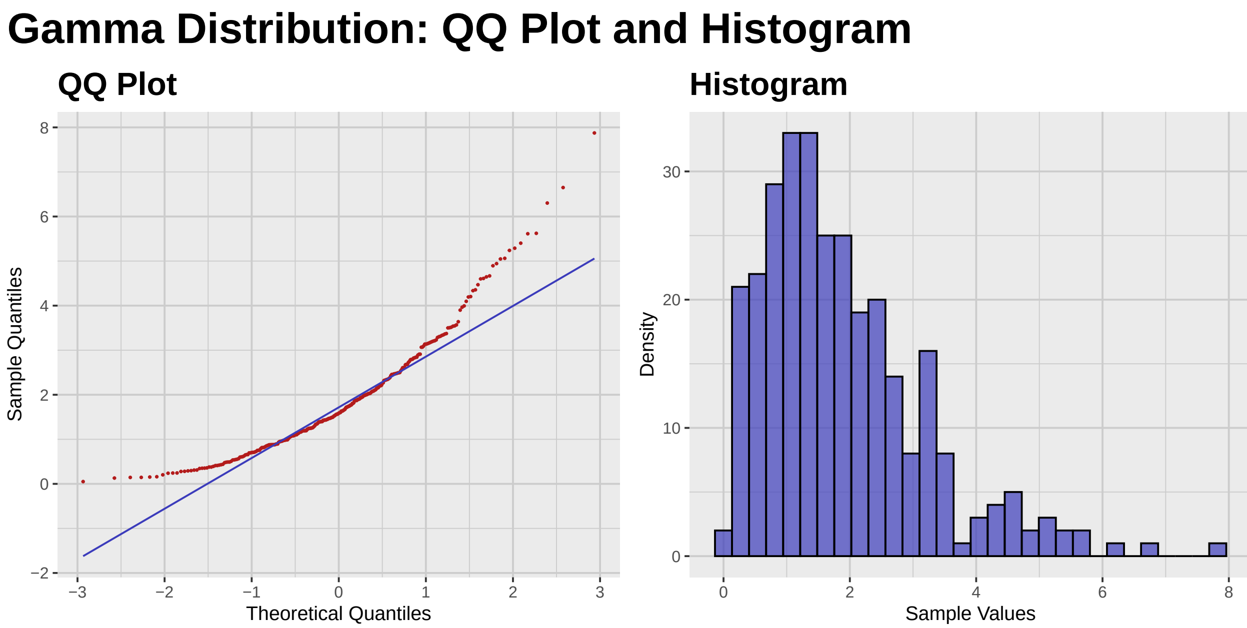


Figure 2.3: QQ plot and histogram of the Gamma distribution samples.

We will now construct the QQ plot and histogram for the Laplace distribution samples:

```
laplace_qq_plot <- laplace_sample_tbl %>%
  ggplot(aes(sample = sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plot",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

laplace_histogram <- laplace_sample_tbl %>%
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
```

```
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()
laplace_plot <- laplace_qq_plot + laplace_histogram +
  plot_layout(ncol = 2) +

  plot_annotation(
    title = "Laplace Distribution: QQ Plot and Histogram",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
laplace_plot
```

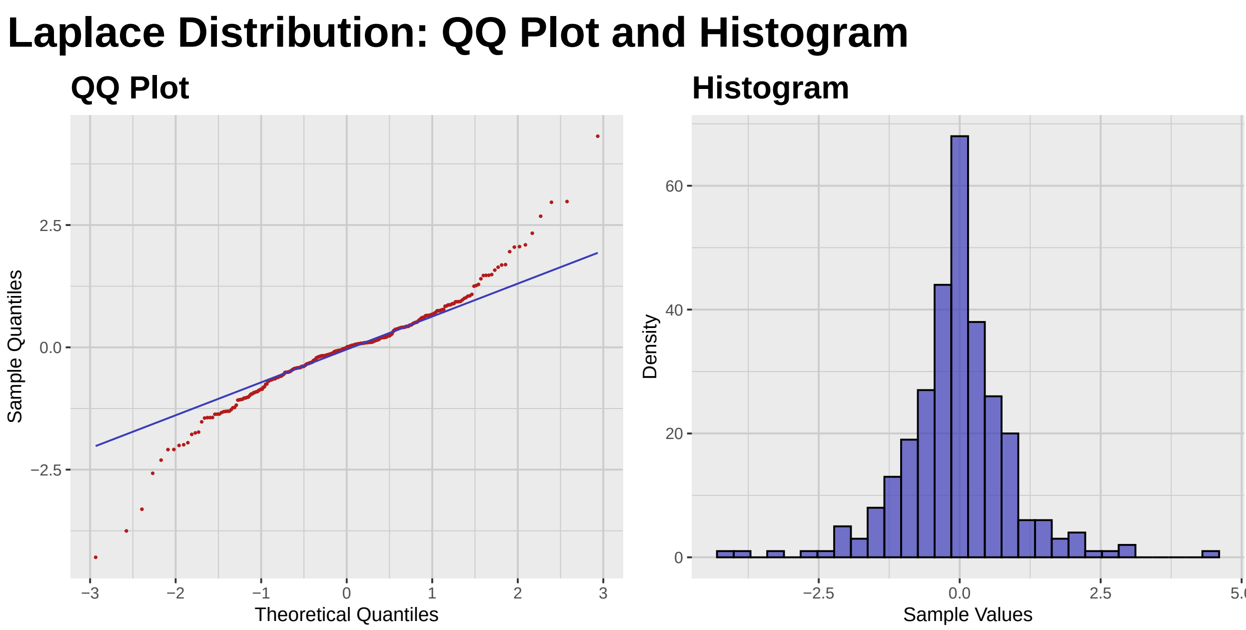


Figure 2.4: QQ plot and histogram of the Laplace distribution samples.

## 2.4 Sample Skewness and Kurtosis

So far we have defined skewness and kurtosis for random variables. However, in practice, we often work with samples rather than entire populations. Therefore, we need to define sample skewness and sample kurtosis.

**Definition 2.4.1.** Let $x_1, x_2, \ldots, x_n$ be a sample of size $n$. The *sample mean* $\bar{x}$ and *sample variance* $s^2$ are given by

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2.$$

The *sample skewness* $g_1$ is given by

$$g_1 = \frac{n}{(n-1)(n-2)}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^3.$$

The *sample excess kurtosis* $g_2$ is given by

$$g_2 = \frac{n(n+1)}{(n-1)(n-2)(n-3)}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{s}\right)^4 - \frac{3(n-1)^2}{(n-2)(n-3)}.$$

As an example, we will compute the sample skewness and sample excess kurtosis of the Gamma and Laplace samples we generated earlier. For the Gamma sample we have:

```r
# Compute sample skewness and excess kurtosis for Gamma distribution
gamma_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness        <dbl> 1.293772
$ sample_excess_kurtosis <dbl> 2.164353
```

For the Laplace sample we have:

```r
# Compute sample skewness and excess kurtosis for Laplace distribution
laplace_sample_tbl %>%
  summarise(
    sample_skewness = skewness(sample, type = 2),
    sample_excess_kurtosis = kurtosis(sample, type = 2)
  ) %>%
glimpse()
```

```
Rows: 1
Columns: 2
$ sample_skewness        <dbl> -0.07876743
$ sample_excess_kurtosis <dbl> 3.726374
```

## 2.5 Scaling and Standardization

In many situations, it could happen that the units we are using to measure different variables in our data are not compatible. For example, we could have a dataset that contains the height of individuals in centimeters and their weight in kilograms. In such cases, it is often useful to *scale* the variables so that they are comparable. In this section, we will discuss the most common methods for scaling and standardizing random variables and samples.

### 2.5.1 Scaling Random Variables

**Definition 2.5.1.** An *affine transformation* of a random variable $X$ is a transformation of the form

$$Y = aX + b$$

where $a$ and $b$ are constants.

*Remark* 2.5.2. In the context of statistics, affine transformations are often refered to as *scaling* (even when they also include a translational component).

Now assume that $X$ is a random variable with mean $\mu$ and standard deviation $\sigma$. Then the random variable

$$Y = aX + b$$

will have mean $a\mu + b$ and standard deviation $|a|\sigma$. In particular, if we choose $a = 1/\sigma$ and $b = -\mu/\sigma$, then the random variable

$$Z = \frac{X - \mu}{\sigma}$$

will have mean 0 and standard deviation 1. This particular affine transformation is called a *standardization*, and the resulting random variable $Z$ is called the *standardized* version of $X$. $Z$ is also sometimes called the *z-score* of $X$.

*Remark* 2.5.3. Notice that if $X$ is not normally distributed, then $Z$ will *not* be the standard normal distribution. Standardization does not make a non-normal distribution normal.

*Observation* 2.5.4. Notice that, by definition,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3} = \gamma_1(X)$$

and

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

As an example, we will consider again the Gamma distrubution that we introduced earlier. In this case we have that

$$\mu = \frac{\alpha}{\beta} \quad \text{and} \quad \sigma = \frac{\sqrt{\alpha}}{\beta}.$$

Using this information, we can generate the plot of standadized version of the Gamma distribution and compare it with the original one.

```
# Calculate density values
z_gamma_distribution_tbl <- gamma_distribution_tbl %>%
  mutate(
    z_data = (x_data - gamma_mean) / gamma_sd,
    z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd
  ) %>%
  select(z_data, z_density)

# Calculate mean and median
z_gamma_mean <- 0
z_gamma_median <- (gamma_median-gamma_mean) / gamma_sd

# Plot
z_gamma_distribution_plot <- z_gamma_distribution_tbl %>%
ggplot(aes(x = z_data, y = z_density)) +
  geom_line(
    color = c_palette["C blue"],
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_mean,
    color = c_palette["C red"],
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = z_gamma_median,
    color = c_palette["C green"],
    linetype = "dotted",
    linewidth = 1
  ) +
  annotate(
    "text",
    x = z_gamma_mean,
    y = 0.55,
    label = "Mean",
    color = c_palette["C red"],
    hjust = -0.1
  ) +
  annotate(
    "text",
    x = z_gamma_median,
    y = 0.55,
    label = "Median",
    color = c_palette["C green"],
```

```r
    hjust = 1.1
  ) +
  coord_cartesian(ylim = c(0, 0.6)) +
  labs(
    title = paste(
      "Standardized"
    ),
    x = "X",
    y = "Density"
  ) +
  theme_krul()

gamma_distribution_plot <- gamma_distribution_plot +
  labs(
    title = "Original"
  )

combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Gamma Distribution: Original and Standardized",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

combined_gamma_plot
```

# Gamma Distribution: Original and Standardized

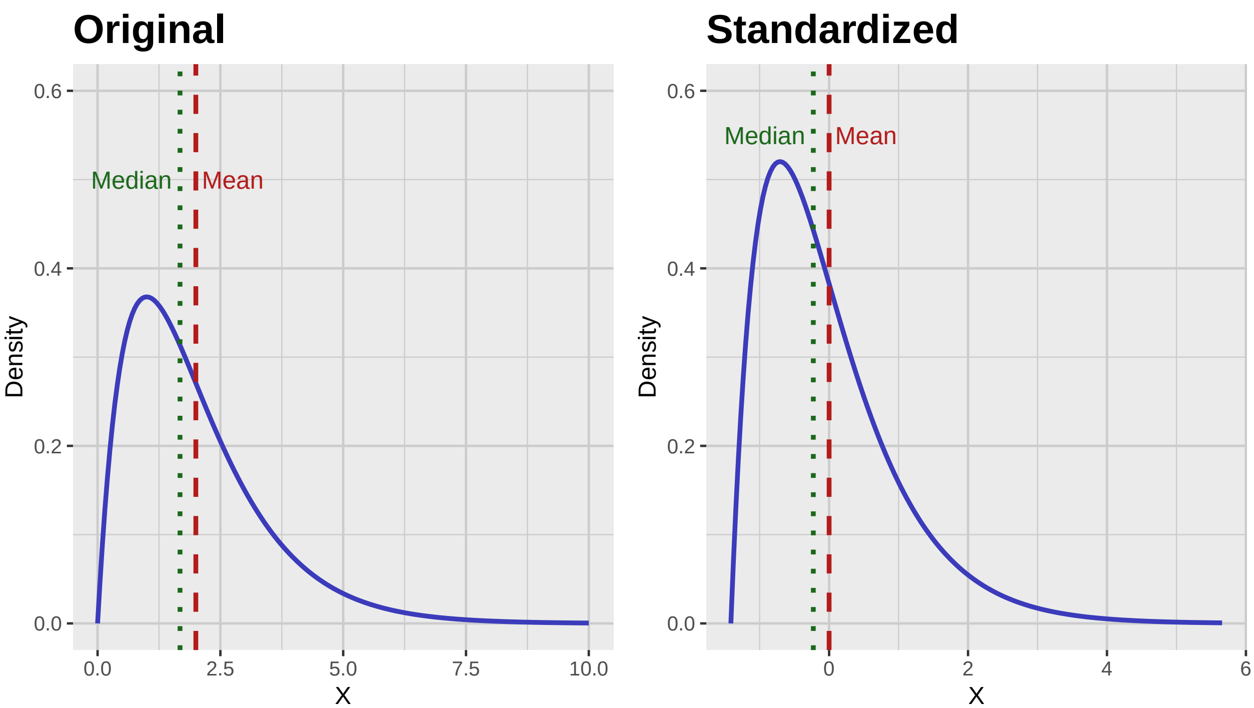


Figure 2.5: Comparison of the probability density function (PDF) of the original and standardized Gamma distribution.

### 2.5.2 Scaling Samples

For a given a sample, it is not always known what the underlying distribution is. In particular, we may not know what the mean and the standard deviation of the underlying population are. However, we can still try to scale our data but we should keep in mind that in this case there is more than one scaling method. The most common are:

1. *Standardization*: This method uses the sample mean and sample standard deviation to scale the data. The standardized sample is given by

$$T = \frac{X - \bar{X}}{S},$$

where $\bar{X}$ is the sample mean and $S$ is the sample standard deviation.

2. *Min-Max Scaling*: This method scales the data to a fixed range, usually $[0, 1]$. The scaled sample is given by

$$T = \frac{X - \min(X)}{\max(X) - \min(X)},$$

where $\min(X)$ and $\max(X)$ are the minimum and maximum values of the sample, respectively. This method is very sensitive to outliers, as it uses the minimum and maximum values of the sample.

3. *Robust Scaling*: This method scales the data using the median and interquartile range (IQR). The scaled sample is given by
$$T = \frac{X - \text{median}(X)}{\text{IQR}(X)},$$
where $\text{IQR}(X) = Q_3 - Q_1$ is the interquartile range of the sample. This method is more robust to outliers than min-max scaling, as it uses the median and IQR instead of the minimum and maximum values.

4. *Normalization*: This method simply divides each data point by the corresponding vector norm. The normalized sample is given by
$$T = \frac{X}{\|X\|},$$
where $\|X\|$ is the vector norm of the sample. It is not used as often as the previous methods.

*Remark* 2.5.5. As we mentioned earlier, scaling does not make a non-normal distribution normal. In particular, the sample skewness and sample excess kurtosis of the scaled sample will be the same as those of the original sample.

As an example, we will consider the different scaling methods applied to the Gamma sample we generated earlier.

```
scale_lookup_tbl <- tibble(
  code = c(
    "sample",
    "sample_standard",
    "sample_min_max",
    "sample_robust",
    "sample_normal"
  ),
  label = c(
    "Original Sample",
    "Standardized",
    "Min-Max Scaled",
    "Robust Scaled",
    "Normalized"
  )
)

scaled_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
   sample_standard= standard_scale(sample),
   sample_min_max= min_max_scale(sample),
   sample_robust= robust_scale(sample),
   sample_normal= normal_scale(sample)
  ) %>%
  pivot_longer(
    cols = everything(),
```

```
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) %>%
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl %>%
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~ scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plots",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl %>%
  ggplot(aes(x = sample_scaled)) +
  facet_wrap(
    ~ scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
```

```
  ) +
  labs(
    title = "Histograms",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparison of Scaling Methods Applied to the Gamma Sample",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

scaled_gamma_plot
```

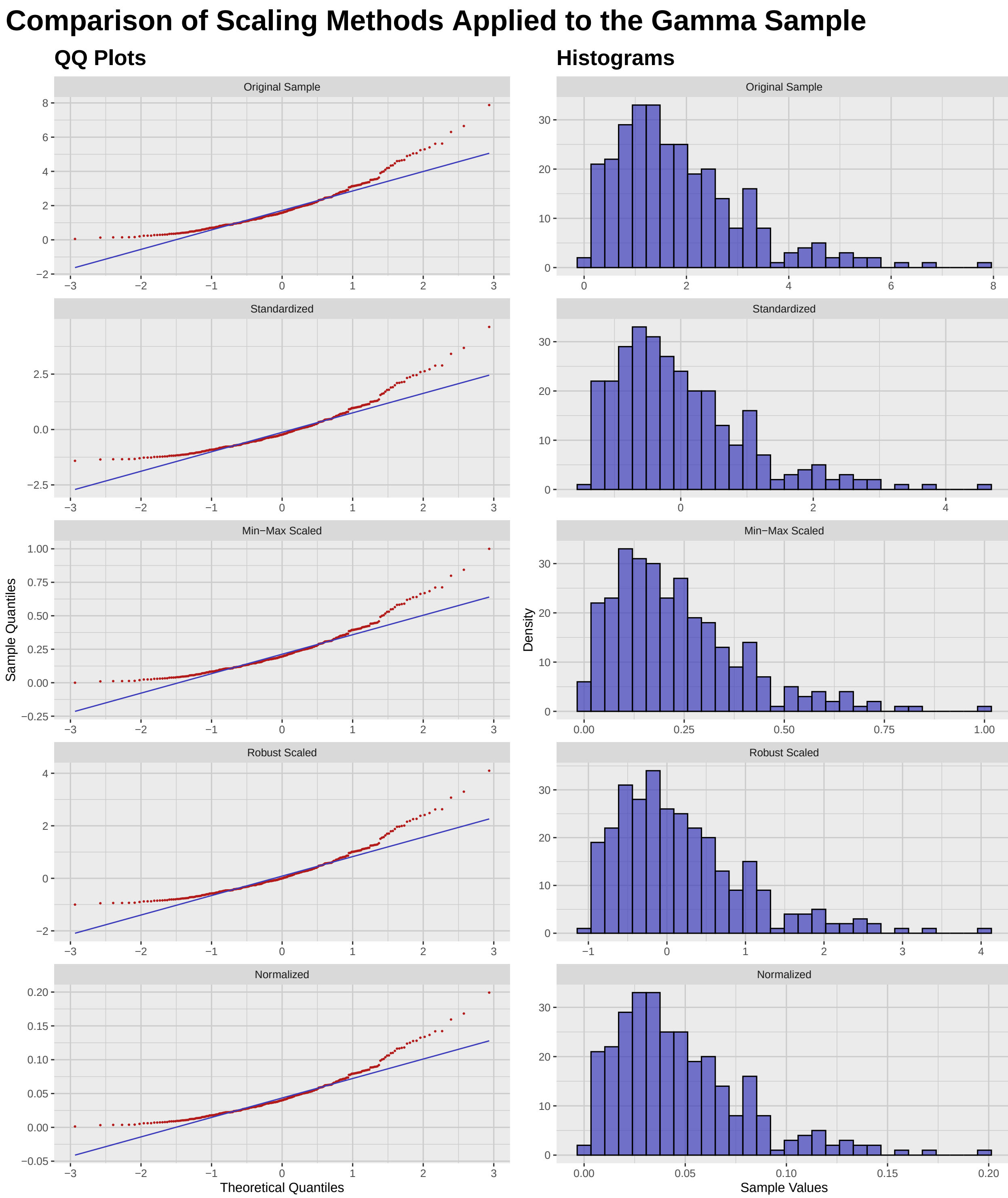


Figure 2.6: Comparison of different scaling methods applied to the Gamma distribution sample. Notice that the QQPlots are just rescaled versions of the original QQ plot, and therefore they all have the same shape.

## 2.6 Activity: Visualizing the Non-Normality of Samples

For each of the following distributions, construct a sample of size 300 and compute their associated sample Skewness and Kurtosis. After that, construct and compare their associated QQ plots and histograms.

1. Exponential distribution with rate parameter $\lambda = 1$.
2. Uniform distribution on the interval $[0, 1]$.
3. Beta distribution with shape parameters $\alpha = 2$ and $\beta = 5$.
4. Cauchy distribution with location parameter $x_0 = 0$ and scale parameter $\gamma = 1$.
5. Chi-squared distribution with degrees of freedom $k = 3$.

# Chapter 3

# Normality Testing

Although QQ plots are a good way to visually assess the normality of a dataset, they can be subjective and imprecise. Therefore, it becomes necessary to develop more objective and precise statistical tests to determine whether the data we are analyzing follows a normal distribution. In this chapter, we will explore some commonly used statistical tests for normality and discuss some transformations that can be performed on our data to make it more normal-like.

## 3.1 Normality Tests

There are several statistical tests that can be used to determine whether a dataset follows a normal distribution. Some of the most commonly used tests include:

1. *Shapiro-Wilk test*: This test is based on the correlation between the data and the corresponding normal scores. It is a powerful test for small sample sizes (less than 50) and is widely used in practice. However, it can be overly sensitive to small deviations from normality in larger sample sizes.

2. *Kolmogorov-Smirnov test*: This test compares the empirical distribution function of the data with the cumulative distribution function of the normal distribution. It is a non-parametric test and can be used for any sample size. However, it is less powerful than the Shapiro-Wilk test for small sample sizes. It is also sensitive to scale and location parameters, which can affect the results of the test.

3. *Anderson-Darling test*: This test is similar to the Kolmogorov-Smirnov test but gives more weight to the tails of the distribution. It is a powerful test for small sample sizes and is widely used in practice.

4. *Jarque-Bera test*: This test is based on the skewness and kurtosis of the data. It is a powerful test for large sample sizes and is widely used in practice. However, it can be overly sensitive to small deviations from normality in smaller sample sizes.

We will now apply these tests to the gamma sample data we created in the previous chapter:

```r
normality_tests_lookup_tbl <- tibble(
  code = c("shapiro", "ks", "ad", "jb"),
  label = c(
    "Shapiro-Wilk",
    "Kolmogorov-Smirnov",
    "Anderson-Darling",
    "Jarque-Bera"
  ),
)

normality_test_gamma_sample_tbl <- gamma_sample_tbl %>%
  summarise(
    shapiro = tidy_shapiro_test(data = ., value_col = sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

The results of the normality tests are as follows:

```r
normality_test_gamma_sample_tbl
```

```
# A tibble: 4 x 2
  test_factor          p_value
  <fct>                  <dbl>
1 Shapiro-Wilk       1.58e-12
2 Kolmogorov-Smirnov 7.93e- 3
3 Anderson-Darling   9.02e-16
4 Jarque-Bera        0
```

As we can see, none of these tests was particularly successful, which is not surprising given that the gamma distribution is not normal.

## 3.2 Transformations

To make our data more normal-like, we can apply various transformations to any given sample. Some of the most common transformations include:

1. *Box-Cox transformation*: This is a family of power transformations that can be applied only to data that is always positive. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(X) & \text{if } \lambda = 0 \end{cases}$$

   where $\lambda$ is some parameter.

2. *Yeo-Johnson transformation*: This transformation is similar to the Box-Cox transformation but can handle negative values. It is useful for data that is positively or negatively skewed. It is given by the formula:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{if } X \geq 0, \lambda \neq 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{if } X < 0, \lambda \neq 2 \\ \log(X+1) & \text{if } X \geq 0, \lambda = 0 \\ \log(-X+1) & \text{if } X < 0, \lambda = 2 \end{cases}$$

   where $\lambda$ is some parameter.

We will now apply the Box-Cox transformation to the gamma sample data with parameters $\lambda = 0$, 0.25, 0.5, 0.75, and 1:

```
box_cox_lookup_tbl <- tibble(
  code = c("box_cox_0", "box_cox_025", "box_cox_05", "box_cox_075", "box_cox_1"),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

box_cox_gamma_sample_tbl <- gamma_sample_tbl %>%
    mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) %>%
  pivot_longer(
    starts_with("box_cox_"),
```

```
    names_to = "lambda",
    values_to = "transformed_sample"
  ) %>%
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = lambda_factor
  )


box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(
    ~ lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plots",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

box_cox_gamma_histogram <- box_cox_gamma_sample_tbl %>%
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(
    ~ lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
```

```
    alpha = 0.7
  ) +
  labs(
    title = "Histograms",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

box_cox_gamma_plot <- box_cox_gamma_qq_plot + box_cox_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Comparison of Box-Cox Transformations with Different Lambda Values",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

box_cox_gamma_plot
```

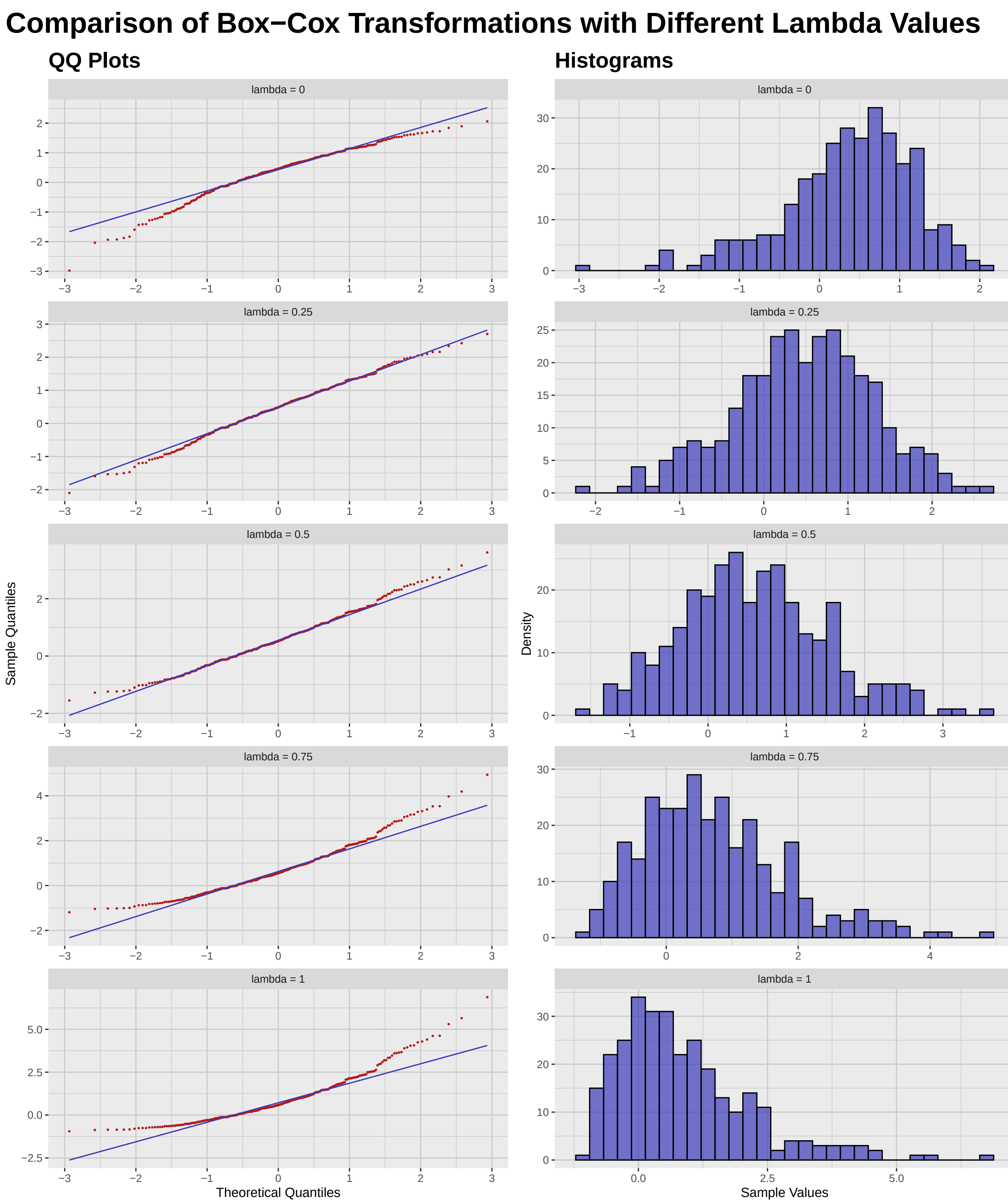


Figure 3.1: Comparison of different Box-Cox transformations applied to the Gamma distribution sample, with parameter lambda = 0, 0.25, 0.5, 0.75, and 1.

## 3.3 Normality Score Optimization

As we saw in the previous section, the Box-Cox transformation can be used to make our data more normal-like. However, the choice of the parameter $\lambda$ is crucial for the success of the transformation. We wish to find the optimal value of $\lambda$ that maximizes the log-likelihood of the transformed data. This can be done using the `powerTransform` function from the `car` package, which performs a Box-Cox transformation and estimates the optimal value of $\lambda$. The following figure shows the log-likelihood profile for the Box-Cox transformation applied to the gamma sample data:

```
# Intercept-only model
fit <- lm(sample ~ 1, data = gamma_sample_tbl)

# Estimate lambda
bc <- powerTransform(fit)

# Extract log-likelihood profile
bc_prof <- boxCox(fit, lambda = seq(-2, 2, 0.1), plotit = FALSE)

# Convert to tibble for ggplot
df <- tibble(lambda = bc_prof$x, loglik = bc_prof$y)

ggplot(df, aes(x = lambda, y = loglik)) +
  geom_line(color = c_pal("C blue"), linewidth = 1) +
  geom_vline(xintercept = bc$lambda, linetype = "dashed", color = c_pal("C red")) +
  labs(title = "Box-Cox Transformation",
       x = expression(lambda),
       y = "Log-likelihood") +
  theme_krul()
```

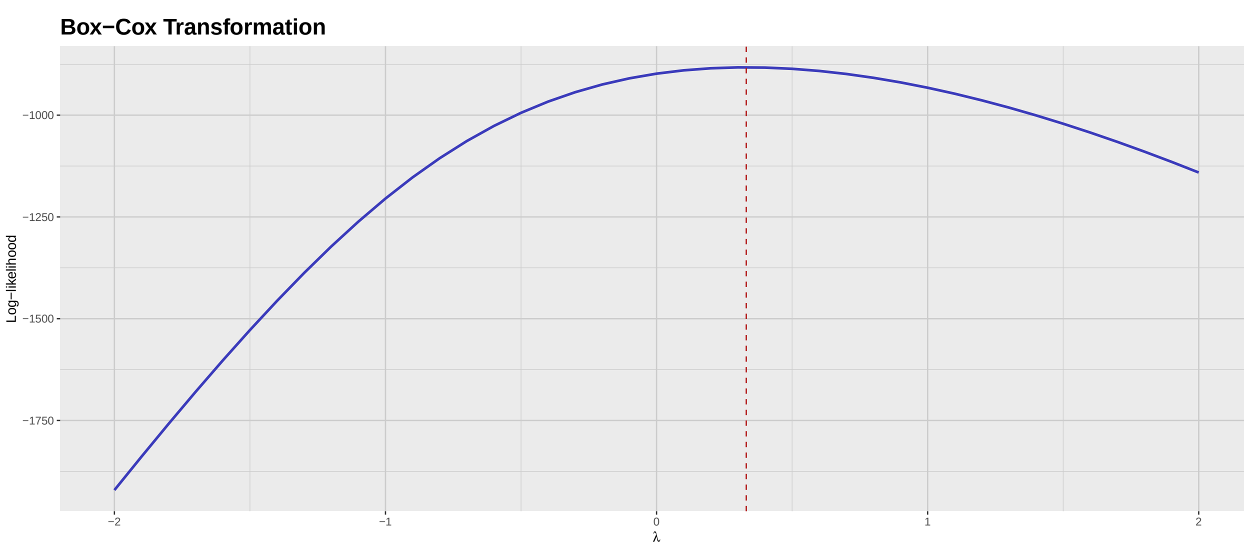


Figure 3.2: Log-likelihood profile for the Box-Cox transformation applied to the Gamma distribution sample. The dashed line indicates the optimal value of lambda.

Finally, we can extract the optimal value of $\lambda$ and transform the data using this value:

```
optimal_lambda <- bc$lambda

optimal_gamma_sample_tbl <- gamma_sample_tbl %>%
  mutate(
    transformed_sample = bcPower(sample, lambda = optimal_lambda)
  )

optimal_gamma_sample_tbl %>%
  summarise(
  shapiro = tidy_shapiro_test(data = ., value_col = transformed_sample)$p_value,
    ks = tidy_ks_test(data = ., value_col = transformed_sample)$p_value,
    ad = tidy_ad_test(data = ., value_col = transformed_sample)$p_value,
    jb = tidy_jb_test(data = ., value_col = transformed_sample)$p_value
  ) %>%
  pivot_longer(
    everything(),
    names_to = "test",
    values_to = "p_value"
  ) %>%
  convert_codes_to_factor(
    code_col = test,
    lookup_tbl = normality_tests_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = test_factor
  ) %>%
  select(test_factor, p_value)
```

```
# A tibble: 4 x 2
  test_factor        p_value
  <fct>                <dbl>
1 Shapiro-Wilk         0.974
2 Kolmogorov-Smirnov   1.000
3 Anderson-Darling     0.977
4 Jarque-Bera          0.878
```

As we can see, the Box-Cox transformation with the optimal value of $\lambda$ significantly improves the normality of the data, as all the normality tests now yield high $p$-values. We now show the QQ plot and histogram of the transformed data:

```
optimal_gamma_qq_plot <- optimal_gamma_sample_tbl %>%
  ggplot(aes(sample = transformed_sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
```

```
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "QQ Plot",
    x = "Theoretical Quantiles",
    y = "Sample Quantiles"
  ) +
  theme_krul()

optimal_gamma_histogram <- optimal_gamma_sample_tbl %>%
  ggplot(aes(x = transformed_sample)) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogram",
    x = "Sample Values",
    y = "Density"
  ) +
  theme_krul()

optimal_gamma_plot <- optimal_gamma_qq_plot + optimal_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Optimal Gamma Distribution Transformation",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )

optimal_gamma_plot
```

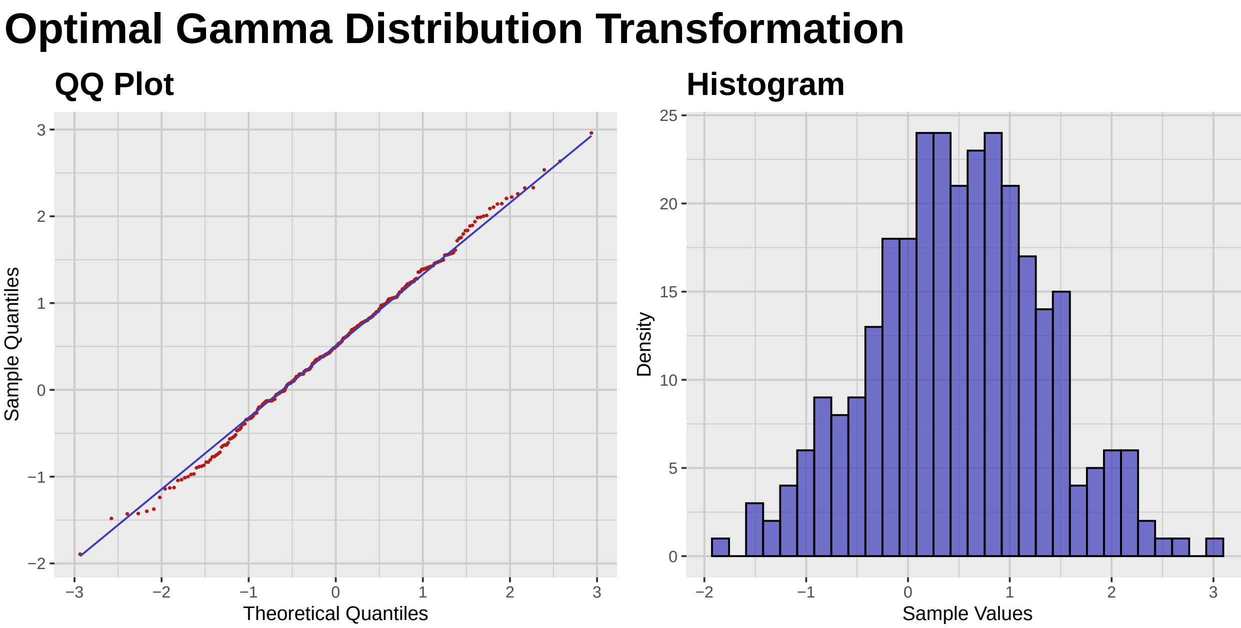


Figure 3.3: QQ plot and histogram of the Optimal Gamma Distribution Transformation. Notice how both plots show a much better fit to the normal distribution compared to the original gamma sample.

# Part II

# Principal Component Analysis

# Chapter 4

# Introduction to Multivariate Analysis

In this chapter we will introduce some of the most basic concepts in multivariate analysis, specifically focusing on the mean vector and covariance matrix of a random vector. These concepts are fundamental in understanding the relationships between multiple variables in a given dataset.

## 4.1 The Mean Vector and the Covariance Matrix

**Definition 4.1.1.** Let $\Pi$ be a population and let

$$X : \Pi \to \mathbb{R}^p$$

be a random (row) vector. The *mean vector* of $X$ is defined as

$$\mu = \mathbb{E}[X] = (\mathbb{E}[X_1], \mathbb{E}[X_2], \dots, \mathbb{E}[X_p]) \tag{4.1}$$

where $X_i$ is the $i$-th component of the random vector $X$. The *covariance matrix* of $X$ is defined as

$$\Sigma = \mathrm{Cov}(X) = \mathbb{E}[(X - \mu_X)(X - \mu_X)^T] = (\mathrm{Cov}(X_i, X_j))_{i,j=1}^p \tag{4.2}$$

where $\mathrm{Cov}(X_i, X_j) = \mathbb{E}[(X_i - \mathbb{E}[X_i])(X_j - \mathbb{E}[X_j])]$ is the covariance between the $i$-th and $j$-th components of the random vector $X$.

*Remark* 4.1.2. Remember that $\mathrm{Cov}(X_i, X_j) = \mathrm{Cov}(X_j, X_i)$, so the covariance matrix is symmetric. Furthermore, $\mathrm{Cov}(X_i, X_i) = \mathrm{Var}(X_i)$, which is the variance of the $i$-th component of the random vector $X$. For this reason, this matrix is also known as the *variance-covariance matrix* of the random vector $X$.